Zero-knowledge proof

[TaSK]

juni 2026

1 Innledning

Dette tekniske notatet gir en oversikt over en publisert artikkel [2, 3]. Fordi dette notatet beskriver en artikkel er her ingen nye resultater. Termer som "artikkelen", "kapittel 2", "figur 2", og tilsvarende refererer til [3]. Der det er nødvendig å referere til denne teksten bruker vi eksplisitt "i dette notatet".

Det gjøres ingen antagelser om kanaler som meldinger sendes på, utover ordning i tid (en melding ankommer ikke føær den er sendt). I særdeleshet antas det ikke at kanaler tilbyr konfidensialitet.

2 Bakgrunn

Tema er hvordan Alice kan overbevise Bob at hun kjenner en hemmelighet, uten å hverken fortelle noe av verdi til Bob (ut over at hun kjenner hemmeligheten), og heller ikke til Charlie (som ikke kjenner hemmeligheten) som lytter på samtalen.

For eksempel: Gitt at Alice har konfidensielle data, for eksempel et dokument; la oss kalle dokumentet for D. Hun tilbyr ett eller annet til andre som også har dokumentet. Bob mener han har det, og ønkser å bevise til Alice om det. Når følgende protokoll er ferdig er A overbevist om at B også har (tilgang til) D:

$A \longrightarrow B : N'_A$

$B \longrightarrow A : H\{N'_A \,.\, D\}$

hvor notasjonen . betyr sammensetting (*concatination*), N'_A "et svært stort tilfeldig tall" (la oss si $\approx 2^{256}$), og $H\{m\}$ et digitalit fingeravtrykk (*hashing*) av m.

Fordi han lytter på all kommunikasjon kjenner Charlie nå N'_A og $H\{N'_A \,.\, D\}$. Det vil si at om han har stjålet et dokument D' kan han undersøke om $D' = D$ ved å beregne $H\{N'_A \,.\, D'\}$ og sammenligne med $H\{N'_A \,.\, D\}$. Det vil at protokollen mellom A og B lakk (meta)informasjon om D til C.

Evnen til å kunne verifisere om $D = D'$ er en betydelig forde som kan gi næring til mange typer angrep. *Zero Knowledge* handler om å overbevise den andre uten å lekke informasjon.

3 Matematisk utgangspunkt

Den matematiske konteksten i artikkelen er språk og kompleksitetsklasser; vi gir derfor en kort oppfriskning.

3.1 Språk

Alan Turing var den første til å representere (matematiske) spørsmål med symboler (tekst) [5] men det var først 25 år senere at det ble forstått at dersom et problem representeres som en kompakt streng (sekvens av symboler) så vil lengden av strengen være et mål på "størrelsen" av problemet [4]. For en introduksjon til anbefaler vi [1]; første kapittel går nøye gjennom representasjon av problemer.

I spørsmål om involverer tall er det åpenbart hvordan problemet skal representeres. For eksempel: Om spørsmålet er: "Er tallet et primtall?" så er problemet som skal besvares et tall. Tallet (problemet) kan være $X = 323$ (som vil si $(101000011)_2$). Lengden på dette problemet er 9.

Når spørsmålet omhandler en graf, for eksempel om grafen er forbundet, kan kan vi konvertere grafen til en matrise (hvor $M_{ij} = 0$ om det ikke er en kant mellom nodene i og j). Når en slik matrise skrives ut blir resultatet igjen en streng (av 0 og 1).

Når vi bruker strenger til å representere problemer på denne måten kan vi betrakte strengene som et "ord"; hver streng representerer ett (og bare ett) problem. Indirekte har vi nå også fått et mål på størrelse: Jo lenger streng jo større problem.

Om vi ser på de to uendelige settene av alle probløemer (tall eller fgrafer) så vil spørsmålet ("Er tallet prim?", eller "Er grafen forbundet?") dele settet i to: Ja eller nei; de begge tilfellene vil to sub-settene ikke være like store. Settet som representerer "ja", det er ord i *et språk*.

Spørsmålet om X er prim, det heter PRIMES. Den uformelle beskrivelksen blir at gitt en binær streng som vi tolker som et tall X slik at $X \in \mathbf{N}$, er X et primtall? Her behøver vi heldigvis ikke definere hva primtall er, for det vet vi. Språket PRIMES er settet med (binære) strenger som representerer tall som er prim. Akkurat som ikke alle bokstavkombinasjoner tilhører språket norsk, tilhører ikke alle tall språket PRIMES.

Spørsmålet om en graf inneholder en komplett sub-graf (klikk) det heter CLIQUE. Språket L_{CLIQUE} er "ord" hvor svaret er "ja":

$$L_{\text{CLIQUE}} = \{\langle G, k\rangle \mid G \text{ har en klikk med størrelse } k\} \tag{1}$$

Her er G en streng som representerer en graf og k et tall (størrelsen på klikken).

3.2 Kompleksitesklasser

3.2.1 Klassen P

De språkene som kan konstrueres i tid og rom som ikke vokser mer enn polynomisk relativ til lengden på spørsmålet, de sies å tilhøre klassen **P**.

For eksempel PARTALL:

$$PARTALL : \{n : n\text{er partall} \tag{2}$$

Algoritmen for å bestemme om et tall er i PARTALL er å se om siste bit er 0 eller 1. Det tar fast tid (uavhengig av lengen) og derfor klart mindre enn polynomisk tid.

Gitt en graf G kan vi bruke bredde-først eller dybde-først for å finne om én node er forbundet med en annen. Ingen av dem vokser mer enn polynomisk når størrelsen på G er målet. Det vil si at språket STI er i **P**:

$$\text{STI} = \{(G, a, b) : \text{Det er en sti fra } a \text{ til } b \text{ i } G \tag{3}$$

3.2.2 Klassen NP

En annen klasse, som inkluderer settet **P**, er **NP**. Språk i denne klassen er de hvor det er mulig å verifisere et bevis[1] på at en instans har "ja"-svar i polynomisk tid.

For å se at P er innholdt i NP er det tilstrekkelig å notere at elementene i P kan konstrueres i polynomisk tid. Det vil si det ikke er behov for noe bevis i det hele tatt; man kan bare regne ut løsningen.

Et språk som ofte brukes som eksempel er SAMMENSATT:

$$\text{SAMMENSATT} : \{n : \text{n er sammensatt (ikke prim)}\} \tag{4}$$

Et bevis for at n er sammendatt (et "ja"-svar) er $n = xy$. Kompleksiteten ved å verifisere, altså multiplisere, øker med lengden på x og y (som naturligvis øker med lengden på n), men kun polynomisk. Altså er SAMMENSATT i **NP**. Kjernen er at det som skal gjøres er ikke å finne ut om et tall er sammensatt, men undersøke om et bevis er gyldig.

PRIMES er også i **NP** fordi det finnes et bevis somk kan verifiseres i polynomisk tid. Dersom n er prim da vil det finnes en $a^{n-1} = 1(= n)$ og for hver primfaktor q av $n-1$ vil $a^{(n-1)/q} \neq 1(= n)$. Et bevis for at n er prim vil være a sammen med faktoriseringen av $n-1$ (og tilsvarende bevis for at de faktorene er prim).

Her er det viktig å huske at å finne ut om n er prim er ett problem mens avgjøre om beviset på at n er prim er gyldig, er et annet. Kravet for at et problem er i **NP** er altså at det finnes et bevis (og ikke nødvendigvis at beviset er "lett" å finne).

4 Notasjon

References

[1] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 1st edition, 2009.

[2] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, STOC '85, pages 291–304. ACM, 1985.

[3] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[1] På engelsk har *proof* og *evidence* forskjellig betydning. Når vi snakker om *bevis* her mener vi *evidence*.

[4] Juris Hartmanis and Richard E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965.

[5] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1937.